

Cross-platform capability coverage

Contents

Tested on — what’s actually been run (2026-07-04)	1
The backbone (why it’s portable)	1
Legend	2
Notes on the [!] rows (honest limitations)	3
The “Flask” (headless / web server) axis	3
System dependencies (the binaries pip can’t install)	3
Verifying on your platform	4

ELI is **model-, user-, and hardware-agnostic** and runs on **Linux, macOS, Windows, and Android/Termux**, plus **headless / via the web server (FastAPI)**. This document is the honest map of how each capability family behaves on each target.

Tested on — what’s actually been run (2026-07-04)

The matrix below is **code coverage** — how each capability is *implemented* per OS. This bit is separate and more honest: what I’ve genuinely **run**.

- **Run-verified, end to end: Linux (x86_64) + NVIDIA.** Install, first-run, the full test suite (7,300+ passing), voice, vision, the server — all of it. This is the tested path.
- **Code-verified, not yet run on real hardware: AMD (ROCm), Windows, macOS, Android.** The installers, per-OS requirements, paths, GPU detection (NVIDIA / AMD / Apple), and the abstraction layer below are all present and read line-by-line — but “correct in the code” isn’t the same as “confirmed on the metal,” and I won’t pretend it is. Run ELI on one of these and send me the result (working or broken) — that’s the fastest way to close the gap for everyone.

A few edges worth knowing going in: - **AMD voice is CPU-only** — the speech-to-text engine (CTranslate2) has no ROCm support, so on an AMD GPU it stays on the CPU (works, just not accelerated). The main model + vision use the AMD GPU via hipBLAS. - **Windows secret-file permissions are weaker** — 0600 is a Unix bit Windows ignores; the token/key files lean on %APPDATA% being private (it is, by default). A proper Windows ACL is on the list. - **Big models can be slow to load or run out of memory** — a large model takes a minute on first use, and if VRAM/RAM is tight the server can drop mid-reply. Run the server on its own (not alongside the desktop app) so the model loads once.

The backbone (why it’s portable)

Every OS-touching operation routes through one of three cross-platform layers — never a raw Linux call: - **eli/utils/platform_compat.py** — open URL/file/app, notifications, clipboard, volume/mute, play-sound, key/type, user dirs. Branches for Linux / macOS / Windows / Android(Termux) / BSD, each guarded with shutil.which and degrading gracefully. - **eli/core/paths.py** — all data/config/cache/log/models paths via platformdirs (XDG on Linux, ~/Library on macOS, %APPDATA%/LOCALAPPDATA% on Windows, sdcard on Android). - **eli/perception/os_controller.py / eli/system/portable_app_control.py** — screenshots, app enumeration/control with per-OS implementations.

No capability hard-crashes off Linux — anything platform-specific is guarded and returns a clean “unavailable” result instead of raising.

Legend

[yes] full · [!] Linux/desktop-primary, graceful degrade elsewhere · needs a desktop (display/ input) — not on a headless server or Android · ☐ N/A on that platform

Capability family	Linux	macOS	Windows	Android/Termux	Headless / Web API
Chat, reasoning, memory, RAG, KG, User Model	[yes]	[yes]	[yes]	[yes]	[yes]
Web search / news synthesis	[yes]	[yes]	[yes]	[yes]	[yes]
Code examine / repair / file analysis	[yes]	[yes]	[yes]	[yes]	[yes]
File ops + paths (data/config/models)	[yes]	[yes]	[yes]	[yes]	[yes]
Open URL / file / app	[yes]	[yes]	[yes]	[yes] (termux)	[yes] (URL/file)
Desktop notifications	[yes]	[yes]	[yes] (plyer)	[yes] (termux)	☐
Clipboard get/set	[yes]	[yes]	[yes]	[yes] (termux)	☐
Volume / mute	[yes] (pactl)	[yes] (osascript)	[yes] (pycaw)	☐	☐
Play sound	[yes]	[yes]	[yes]	[yes] (termux)	☐
Screenshot	[yes]	[yes]	[yes]		☐
Type text / key press	[yes]	[yes] (pyautogui)	[yes] (pyautogui)		☐
Voice STT / TTS	[yes]	[yes]	[yes]	[!] (audio dev)	☐
Media control (play/pause/next)	[yes] (playerctl)	[!]	[!]	[!] (termux)	☐
Window management (focus/move/tile)	[yes] (wmctrl)	[!]	[!]	☐	☐
Gaze cursor / AR avatar	[yes]	[!]	[!]	☐	☐
GUI (PySide6 desktop app)	[yes]	[yes]	[yes]	☐	☐
Web app / REST API (FastAPI server)	[yes]	[yes]	[yes]	[yes] (client)	[yes]

Capability family	Linux	macOS	Windows	Android/Termux	Headless / Web API
Scheduling, habits, proactive, reflection, self-improve	[yes]	[yes]	[yes]	[yes]	[yes]
LoRA fine-tuning (training/)	[yes]	[!] (CPU/MPS)	[yes]	☐	[yes]

Notes on the [!] rows (honest limitations)

- **Media control** uses playerctl (MPRIS) on Linux; on macOS/Windows/Android it falls back to best-effort and may report “no player” rather than controlling a specific app. (Media *playback* via `play_sound` is fully cross-platform.)
- **Window management** (`wmctrl/xdotool`) is X11/Linux-centric; on macOS/Windows these actions degrade gracefully (return “not available”) — they don’t crash.
- **Gaze/AR** needs a camera + display and `opencv`; desktop-only, Linux-tested.
- These are *guarded* — they never raise; they return a clean unavailable result and ELI says so.

The “Flask” (headless / web server) axis

The FastAPI server (`api/server.py`, see `docs/SERVER_AND_WEB_APP.md`) exposes `POST /v1/chat` and `POST /v1/execute`. Through it, **every capability that doesn’t require a local display or input device works** — chat, memory, search, code, file ops, scheduling, status, open-URL. The rows (screenshot, type, gaze, window-mgmt, GUI) inherently need a desktop session and so are not available over a remote/headless API — by nature, not by bug. A phone talking to a desktop-hosted server gets the full *assistant*; the desktop-control actions run on the host.

System dependencies (the binaries pip can’t install)

Core ELI — chat, memory, RAG, GGUF inference, the GUI, and the web server — needs only the pip packages and works right after install with no system tools. Some *optional* features need OS-level binaries; the installers handle these best-effort:

Feature	System dep	Linux (install.sh)	macOS (install.sh)	Windows (install.ps1)
Screen OCR	tesseract	apt/dnf/pacman	brew	winget (UB-Mannheim)
Voice input (mic)	PortAudio	apt/dnf/pacman	brew	bundled in the PyAudio wheel
Media playback / Whisper	ffmpeg, mpv	apt/dnf/pacman	brew	winget (FFmpeg)
Desktop control	xdotool, wmctrl, scrot, xclip/wl-clipboard, libnotify	apt/dnf/pacman	native APIs	native APIs
GPU offload	CUDA / AMD ROCm / Metal	CUDA --install-cuda; AMD auto-detected → ROCm/hipBLAS build	Metal (built in)	CUDA / cuda (winget)

Feature	System dep	Linux (install.sh)	macOS (install.sh)	Windows (install.ps1)
Web automation	playwright browsers	playwright install	playwright install	playwright install
Volume / clipboard / notifications	—	system tools above	native (osascript/pbcopy)	native (pycaw/clip/plyer)

The installer installs these **when a package manager + sudo/admin is available**; otherwise it **prints the exact command** so you can run it. None are required for ELI to start — missing ones only disable that one optional feature (gracefully).

Verifying on your platform

- `python -c "from eli.utils.platform_compat import normalize_platform; print(normalize_platform)"` → prints linux/macos/windows/android.
- Each OS-action returns `{"ok": false, ...}` with a clear reason when a backend is missing — so you can see degradation explicitly rather than guessing.